



Gathering of oblivious robots on infinite grids with minimum traveled distance ☆,☆☆



Gabriele Di Stefano^a, Alfredo Navarra^{b,*}

^a Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica, Università degli Studi dell'Aquila, Italy

^b Dipartimento di Matematica e Informatica, Università degli Studi di Perugia, Italy

ARTICLE INFO

Article history:

Received 15 January 2015

Available online 20 September 2016

Keywords:

Distributed computing

Asynchronous system

Look–Compute–Move model

Weber point

Oblivious and anonymous robots

ABSTRACT

A largely studied version of the gathering problem asks to move a set of robots initially placed at different vertices of an anonymous graph toward a common vertex, and to let them remain at such a vertex. Asynchronous robots move based on the so-called *Look–Compute–Move* model. Each time a robot wakes-up, it perceives the current configuration in terms of occupied vertices (Look), it decides whether to move toward one of its neighbors (Compute), and then it performs the computed move (Move), eventually. So far, the main goal has been to detect the minimal assumptions that allow to accomplish the task, without taking care of any cost measure. In this paper, we are interested in optimal algorithms in terms of total number of moves. We consider infinite grids, and we fully characterize when optimal gathering is achievable by providing a distributed algorithm.

© 2016 Published by Elsevier Inc.

1. Introduction

Robot-based computing systems have been largely addressed to the *gathering* (or *rendezvous*) problem either concerning open spaces or discrete representations. A team of robots, initially placed at different locations, have to gather at the same place (not determined in advance) and remain there. Many variants of the problem have attracted the interest of the research community (see e.g., [2–6] and references therein).

In this paper, we consider infinite grids as input graphs where robots are initially placed at different vertices. Robots are assumed to be oblivious (without memory of the past), uniform (running the same deterministic algorithm), autonomous (without a common coordinate system, identities or chirality), asynchronous (without central coordination), without the capability to communicate. Neither vertices nor edges are labeled and no local memory is available on vertices. Robots operate according to the so-called *Look–Compute–Move* cycles [7–11]. In each cycle, a robot wakes-up and takes a snapshot of the current global configuration (Look), then, based on the perceived configuration, decides either to stay idle or to move to one of its adjacent vertices (Compute), and in the latter case it makes an instantaneous move to this neighbor

☆ The work has been supported in part by the Italian Ministry of Education, University, and Research (MIUR), project PRIN 2012C4E3KT “AMANDA – Algorithmics for MAssive and Networked DAta”, and by the National Group for Scientific Computation (GNCS-INdAM).

☆☆ A preliminary version of this paper appeared in *Proceedings of the 16th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS'14)* [1].

* Corresponding author.

E-mail addresses: gabriele.distefano@univaq.it (G. Di Stefano), alfredo.navarra@unipg.it (A. Navarra).

(Move). As moves are instantaneous, robots are always detected on vertices during the Look phase and not on edges. Cycles are performed asynchronously for each robot. This means that the time between Look, Compute, and Move phases is finite but unbounded, and is decided by the adversary for each robot. Hence, robots may move based on significantly outdated perceptions. Robots are oblivious, i.e., they do not have any memory of past observations. Thus, the target vertex (which is either the current position of the robot or one of its neighbors) is decided by the robot during a Compute phase solely on the basis of the location of other robots perceived during the Look phase. Robots are anonymous and execute the same deterministic algorithm. They cannot leave any marks at visited vertices, nor send any messages to other robots.

The problem has been largely studied on ring topologies (see, e.g., [7,9,10,12]), where another assumption has been proven to be necessary in order to allow the accomplishment of the task. Robots are in fact equipped with the so called *multiplicity detection* capability in one of its possible forms. This is the ability of robots to acquire information during the Look phase about the number of robots lying on a same vertex. A robot is always able to detect whether a vertex is empty or occupied but if it is empowered with the *global-strong* multiplicity detection, it is able to perceive the exact number of robots that occupy each vertex. In the *global-weak* version, a robot perceives only whether a vertex is occupied by one robot or if a multiplicity occurs, i.e., a vertex is occupied by an undefined number of robots greater than one. The *local* versions instead of global refer to the corresponding ability of a robot in perceiving the information about multiplicities only concerning the vertex where it currently lies. The relevance of the ring topology is motivated by its completely symmetric structure. It means that algorithms for rings are more difficult to devise as they cannot exploit any topological structure, assuming that all vertices look the same. In fact, the devised algorithms are only based on robots' arrangement and not on topology.

In [13], gathering on finite grids has been fully characterized. In particular, even if the global-strong multiplicity detection is assumed, a configuration remains ungatherable if it is periodic (i.e., the same view can be obtained by rotating the grid around its geometric center of an angle of 90 or 180 degrees) on a grid with at least an even side, or it is symmetric with respect to an axis of symmetry passing through edges. For all the other cases, a gathering algorithm has been provided which does not require any multiplicity detection (the only exception is represented by 2×2 grids with three robots). In this case, the chosen topology plays a central role in the designed algorithms. The main criticism with respect to the results in [13] has been that finite grids admit the existence of special vertices like corners that make the problem solvable even without any multiplicity detection.

In this paper, we are interested in infinite grids, that is no special vertices there exist as there are no borders. This clearly makes the problem more difficult as it was for rings where vertices look all the same and topological structure cannot be exploited. Moreover, in most of the previous results, the aim has been concerning the feasibility of the problem, without taking care of any cost measure for the devised solutions. Here, we are interested in optimal gathering algorithms with respect to the number of moves that robots have to perform.

Recently, in [14], basic results for optimal gathering have been introduced. Namely, gathering has been studied on finite graphs with respect to both its feasibility and the possibility to realize it by means of the minimum number of asynchronous moves performed by robots. The paper also introduces the concept of Weber-point [15] on vertex weighted graphs. A similar concept has been introduced in [16].

A Weber-point for a discrete set of sample points in the Euclidean space is the point minimizing the sum of distances to the sample points. In general, it has been proven in [17] that the Weber point is not expressible as an algebraic expression involving radicals since its computation requires finding zeros of high-order polynomials even for just five points (see also [18]).

On graphs with robots, a Weber-point (WP) is a vertex of the graph that minimizes the sum of the length of the shortest paths from it to each robot. Contrary to what happens in the Euclidean space, on graphs with robots there might occur more than one WP. These are all the vertices of the graph that minimize the sum of the shortest paths from each robot toward each of such vertices. On the positive side, WPs on graphs can be easily computed as long as a robot is empowered with the global-strong multiplicity detection. Without this assumption, in fact, a robot may lose the ability to correctly evaluate the current WPs as soon as a multiplicity is created. If an algorithm is able to assure gathering on a WP by letting the robots move along the shortest paths toward such a vertex, we talk about *optimal* algorithm. This is clearly optimal with respect to the number of asynchronous moves performed by the robots. However, sometimes it is just not possible to gather the robots on a WP, but still a gathering algorithm can be designed.

Our results. In this paper, we fully characterize optimal gathering on infinite grids where the topology does not help in detecting a gathering vertex. Nonetheless, the interest in infinite grids also arises by the fact they represent a natural discretization of the plane. More specifically, we extend the theory about optimal gathering to infinite grids. We detect all the specific configurations where gathering cannot be performed. For all other configurations, we devise a distributed algorithm that exploits the global-strong multiplicity detection and, assures gathering on a Weber-point by letting move robots along the shortest paths toward such a vertex, i.e., our algorithm is optimal in terms of moves.

Outline. In the next section, we introduce the notation used in the paper and give some basic definitions. In Section 3, we first provide few impossibility results, and then an optimal gathering algorithm for all the other cases. Section 4 is devoted to the correctness proof of the proposed algorithm. Finally, in Section 5, we conclude the paper.

2. Definitions and preliminaries

In this section we provide all the basic definitions and notation necessary for the understanding of the proposed results.

Given a graph G and a function $\ell : V \rightarrow \mathbb{N}$ representing the number of robots on each vertex of G , we call (G, ℓ) a *configuration* whenever $k = \sum_{v \in V} \ell(v)$ is bounded and greater than zero. A configuration is *initial* if each robot lies on a different vertex (i.e., $\ell(v) \leq 1 \ \forall v \in V$). A configuration is *final* if all robots are on a single vertex u (i.e., $\ell(u) > 0$ and $\ell(v) = 0, \ \forall v \in V \setminus \{u\}$). The distance $d(u, v)$ between two vertices u, v in V is the number of edges of a shortest path connecting u to v .

An important role for designing a gathering algorithm that leads an initial configuration to a final one is played by *symmetric configurations*. In order to catch symmetric properties of a configuration, we introduce the next definitions.

Two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$ are *isomorphic* if there is a bijection φ from V_G to V_H such that $\{u, v\} \in E_G$ if and only if $\{\varphi(u), \varphi(v)\} \in E_H$.

An *automorphism* on a graph G is an isomorphism from G to itself, that is a permutation of its vertices mapping edges to edges and non-edges to non-edges.

We extend the concept of isomorphism to configurations in a natural way: two configurations (G, ℓ) and (G', ℓ') are isomorphic if G and G' are isomorphic via a bijection φ and for each vertex v in G , $\ell(v) = \ell'(\varphi(v))$. An *automorphism* on a configuration (G, ℓ) is an isomorphism from (G, ℓ) to itself and the set of all automorphisms of (G, ℓ) forms a group that we call *automorphism group* of (G, ℓ) , denoted by $\text{Aut}((G, \ell))$.

Given an isomorphism $\varphi \in \text{Aut}((G, \ell))$, the *cyclic subgroup* of order p generated by φ is given by $\{\varphi^0, \varphi^1 = \varphi, \varphi^2 = \varphi \circ \varphi, \dots, \varphi^{p-1}\}$ where φ^0 is the identity.

If H is a subgroup of $\text{Aut}((G, \ell))$, the *orbit* of a vertex v of G is $Hv = \{\gamma(v) \mid \gamma \in H\}$.

(G, ℓ) is said *asymmetric* if $|\text{Aut}((G, \ell))| = 1$, *symmetric* otherwise.

Definition 1. Let $C = ((V, E), \ell)$ be a configuration. An isomorphism $\varphi \in \text{Aut}(C)$ is called *partitive* if the cyclic subgroup $H = \{\varphi^0, \varphi^1 = \varphi, \varphi^2 = \varphi \circ \varphi, \dots, \varphi^{p-1}\}$ generated by φ has order $p > 1$ and is such that $|Hu| = p$ for each $u \in V'$.

Note that, in the above definition, the orbits Hu , for each $u \in V$ form a partition of V . The associated equivalence relation is defined by saying that x and y are *equivalent* if and only if there exists a $\gamma \in H$ with $\gamma(x) = y$. The orbits are then the equivalence classes under this relation; two elements x and y are equivalent if and only if their orbits are the same; i.e., $Hx = Hy$. Moreover, note that $\ell(u) = \ell(v)$ whenever u and v are equivalent. It follows that any algorithm allowing to move a robot r , implicitly allows to move all the robots occupying vertices equivalent to that occupied by r .

Let an infinite path be the graph $P = (\mathbb{Z}, E)$ with $E = \{\{i, i+1\} : i \in \mathbb{Z}\}$. An infinite grid is defined as the Cartesian product $G = P \times P$. A vertex of the grid is then an ordered pair of integers called *coordinates*.

If we assume the infinite grid embedded in a Cartesian plane, it is not difficult to see that it admits three types of automorphisms and combinations of them: *translations*, that is a shifting of the vertices by applying the same displacement to each vertex; *rotations*, defined by a center and an angle of rotation; *reflections*, defined by a reflection axis which acts as a mirror. In an infinite grid, the center of a rotation can be a vertex, or the center of an edge, or the center of the area surrounded by four vertices, whereas the angle of rotation can be of 90 or 180 degrees. Reflections axis can be horizontal (vertical), passing through vertices or through the middle of edges, or diagonal (45 degrees), passing through vertices. Regarding translations, even if they are possible for infinite grids, they do not belong to any automorphism group of configurations as these are defined for a finite (not null) number of robots. Note that, an infinite number of robots (or no robots at all) is required also when the configuration admits two parallel axis of symmetry, one axis and one center of rotation not lying on the axis, or two distinct centers of rotation. Moreover the automorphism group of a configuration with a finite number of robots is finite.

Definition 2. Given a configuration (G, ℓ) , with $G = (V, E)$, the *centrality* of each $v \in V$, is $c_{G, \ell}(v) = \sum_{u \in V} d(u, v) \cdot \ell(u)$. A vertex $v \in V$ is a *Weber-point* if it has the minimal centrality, that is, $c_{G, \ell}(v) = \min\{c_{G, \ell}(u) \mid u \in V\}$.

Whenever clear by the context, we refer to the centrality of a vertex v simply by $c(v)$. By definition, a Weber-point (WP) is a vertex that has the overall minimal distance from all the robots in the configuration.

Definition 3. Given a configuration $C = (G, \ell)$, $G_{\text{WP}}(C)$ is the subgraph induced by its WPs.

An algorithm that gathers all robots on a WP via shortest paths is *optimum* with respect to the total number of moves. More formally, a gathering algorithm must define the sequence of moves for each robot, leading to a final configuration. A move is the change of the position of a single robot from a vertex u to an adjacent vertex v . This equals to change the configuration from, say (G, ℓ) to (G, ℓ') , where $\ell(w) = \ell'(w) \ \forall w \in V \setminus \{u, v\}$, $\ell'(u) = \ell(u) - 1$ and $\ell'(v) = \ell(v) + 1$.

Let S_C be the minimal (finite) sub-grid containing all the occupied vertices of G , and (S_C, ℓ) be the corresponding configuration. It is worth mentioning that S_C may change while robots move. As a consequence, even though S_C is a

finite grid, the approach of [13] cannot be applied. In fact, the algorithm presented in [13] is strongly dependant on the dimensions of the grid where robots reside.

During the Look phase, a robot perceives (S_C, ℓ) and it is able to recognize its position on S_C if (G, ℓ) is asymmetric. Whereas, if (G, ℓ) admits an isomorphism φ different from the identity, a robot cannot distinguish its position at u from $\varphi(u)$, unless $u = \varphi(u)$. As a consequence, two robots (e.g., one on u and one on $\varphi(u)$) can decide to move simultaneously, as any algorithm is unable to distinguish between them. This fact greatly increases the difficulty to devise a gathering algorithm for symmetric configurations.

If an algorithm allows at least two robots to move concurrently, then there might be a so called *pending* move. This occurs when, due to the asynchrony, one of the robots allowed to move performs its entire Look–Compute–Move cycle while one of the others does not perform the Move phase, i.e. its move is pending. Clearly, all the other robots performing their cycles are not aware whether there is a pending move.

In order to check whether the current configuration could have been obtained from a symmetric one, we introduce the concept of *previous position* for a robot. Sometimes, an algorithm simulates itself by considering a configuration C' which is identical to the current configuration C but for the position of one robot r . If an execution of the algorithm can lead from C' to C then the simulated position of r in C' is called a *previous position* for r . This method will be used to detect possible pending moves when C' is symmetric.

We say that an algorithm *assures* gathering if it achieves the gathering regardless any possible sequence of the moves it allows, and possible simultaneous moves. We propose to measure the efficiency of a gathering algorithm by counting the number of moves that it requires to gather all robots from an arbitrary initial configuration to a single vertex. We say that an algorithm is *optimal* if it achieves the gathering with a number of moves equal to the centrality of a WP in the initial configuration. Of course, this is a lower bound for each algorithm. In general, there might be cases where optimal algorithms do not exist even though gathering can be still accomplished. As we are going to see, this does not occur in infinite grids.

Theorem 1. [14] *Given a configuration $((V, E), \ell)$ with WPs in $X \subseteq V$, a move of a robot toward a WP x gives rise to a configuration $((V, E), \ell')$ with WPs in $X' \subseteq V$ such that:*

- $c_{\ell'}(v) = c_{\ell}(v) - 1$ for each $v \in X$;
- $x \in X'$;
- $X' \subseteq X$.

When the configuration admits a unique WP, the above theorem suggests an optimal gathering algorithm that also exploits concurrency among robots. In fact, regardless other robots, each one can move toward the only WP via the shortest path, until finalizing the gathering. Since we are assuming global-strong multiplicity detection, when there is a unique WP, this is always recognizable even though multiplicities are generated.

Corollary 1. *If a configuration admits only one WP then optimal gathering can be assured.*

In what follows, configurations with one single WP are called of type *S*.

3. Gathering algorithm

In this section, we first provide some impossibility results for the gathering task on infinite grids. Then, an optimal resolution algorithm for the gatherable cases is provided. In particular, given a configuration C , the algorithm applies different strategies according to the number of robots, the number of WPs and their distribution, the dimensions of S_C . Hence, a full characterization of the problem is determined. For the ease of presentation, we prefer to describe the algorithm from a global perspective rather than a local one. This also helps afterward the explanation of the correctness proof.

3.1. Impossibility results

All cases shown to be ungatherable in [13] for finite grids are inherited here, as the absence of borders makes the problem more difficult even though the global-strong multiplicity detection is assumed. In what follows we analyze all the ungatherable configurations.

Theorem 2. [14] *If a configuration admits a partitive automorphism, then it is ungatherable.*

In infinite grids, the above theorem implies that all initial configurations with an axis of symmetry not passing through vertices or admitting a rotational center not coinciding with a vertex, are ungatherable. In fact, all such configurations are partitive with orbits of size at least two, and only those admitting rotations of 90 degrees have orbits of size four.

In what follows we say that a symmetry is *allowed* if it is not partitive.

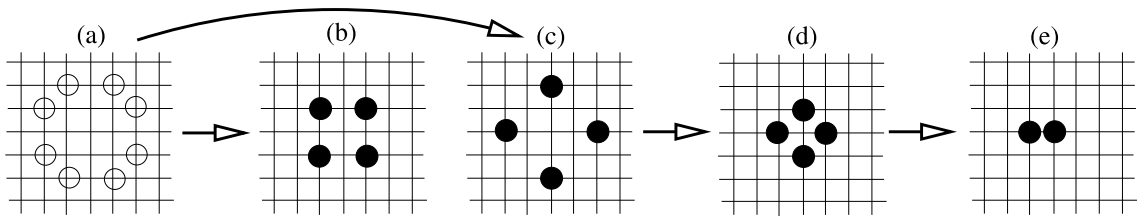


Fig. 1. Empty circles represent single robots; filled circles represent multiplicities. (a) Symmetric configuration with nine WPs in the center. (b) Symmetric configuration obtainable from (a) with nine WPs. (c) Symmetric configuration obtainable from (a) with one WP in the center. (d) Symmetric configuration obtainable from (c) with one WP recognizable only if robots are empowered by global-strong multiplicity detection. (e) Symmetric configuration obtainable from (d) with one WP recognizable only if robots are empowered by global-strong multiplicity detection.

Theorem 3. *If a configuration contains only two robots (or equivalently, two multiplicities of the same size), then it is ungatherable.*

Proof. Assume there exists an algorithm assuring the gathering of two robots and that the adversary prevents simultaneous moves. In order to accomplish the gathering task, robots must reach a configuration where they lie on two adjacent vertices, eventually. In this case, the configuration admits a reflection where the axis passes through the edge between the two occupied vertices, but not on vertices. This configuration admits a partitive automorphism of order two, then by Theorem 2 it is ungatherable.

Similar arguments can be applied when the initial configuration contains two multiplicities of the same size. In fact, when two multiplicities of the same size occur, the adversary can make synchronous all robots belonging to the same multiplicity. Hence, the global behavior of the two multiplicities equals that of two single robots. $\square$

Theorem 4. *If a configuration $C = (G, \ell)$ contains only four robots (or equivalently, four multiplicities of the same size) disposed on the corners of S_C , then C is ungatherable.*

Proof. Assume $C = (G, \ell)$ contains only four robots. If S_C has a side with an even number of vertices, C admits a partitive automorphism of order two, then by Theorem 2 the configuration is ungatherable.

Assume then all the sides of S_C have an odd number of vertices. Since C admits two orthogonal axes of reflections that meet at the central vertex of S_C , any move can be made by all robots since they all look the same. The adversary can make move only two robots for which the movement is performed toward the same direction. This brings to a new configuration C' where $S_{C'}$ has a side with a even number of vertices.

Similar arguments can be applied when the initial configuration contains four multiplicities of the same size. $\square$

From now on we denote by the set $\mathcal{U}$ all initial configurations proved to be ungatherable. The set $\mathcal{U}$ includes all partitive configurations, those with only two robots, and those with four robots disposed as the corners of a rectangle. We will show a gathering algorithm for all the remaining initial configurations. It is worth noting that, differently from the finite grid case where any multiplicity detection capability is unnecessary, it becomes fundamental in this context as shown by the next theorem.

Theorem 5. *If robots are not empowered by any multiplicity detection capability, then the gathering problem is unsolvable on infinite grids.*

Proof. The proof is by induction on the number of robots k . For $k = 2$, it follows from Theorem 3. Assume the statement true for a generic $k - 1$, and consider a gathering algorithm for $k > 2$ robots. Since the aim is to let robots meet at some vertex, there must be a moment in which a multiplicity is created. Since the system is asynchronous, the adversary can always decide to allow only one robot at time to move toward an adjacent vertex occupied by another robot. After this move, the number of vertices occupied by the robots decreases to $k - 1$. As robots are assumed to not detect the multiplicity, they behave like the case of just $k - 1$ robots, as the adversary from now on can make synchronous the two robots on the multiplicity. By the inductive hypothesis gathering is thus impossible. $\square$

It is worth noting that in [13], gathering on finite grids was possible without any multiplicity detection due to the existence of special vertices like corners. Here, we are assuming robots empowered by global-strong multiplicity detection. As shown by Fig. 1, relaxing such an assumption to the global-weak or any local multiplicity detection, makes ungatherable in the optimal way some configurations. In Fig. 1.a, it is shown a symmetric configuration which may lead to two different configurations. Only two moves in fact can be defined in order to gather all robots via shortest paths at one of the nine WPs available. It is easy to check that $G_{WP}(C)$ is constituted by the central subgrid of dimension 3×3 . Any optimal algorithm can allow each robot to move toward either the farthest or the closest robot that shares one coordinate with it, any other move would lead robots away from WPs. Note that, the adversary can make all robots synchronously move since for each

pair of robots there exists a symmetry with respect to which their occupied vertices are equivalent. If all robots move synchronously, in the first case (in the second case, resp.) configuration of Fig. 1.b (Fig. 1.c, resp.) is reached. By Theorem 4, configuration in Fig. 1.b is ungatherable. From configuration in Fig. 1.c, only the move toward the center (the unique WP left) can be allowed. If the adversary makes all the robots move for one step, configuration in Fig. 1.d is obtained. From there, if all the robots but those belonging to only one multiplicity make another step, then configuration in Fig. 1.e is reached. From this configuration, without global-strong multiplicity detection, the two multiplicities are indistinguishable and by Theorem 3 the reached configuration is ungatherable.

3.2. Unidimensional grids

We first consider infinite paths as grids with one row and infinite columns.

Lemma 1. *If the number of robots k is odd, then there exists only one WP. If k is even, then all vertices of the subpath delimited by the central robots (including the vertices where such robots lie) are WPs.*

Proof. When k is odd, there are $\frac{k-1}{2}$ robots on the left and $\frac{k-1}{2}$ on the right of the central robot r . Let $c(v)$ be the centrality of the vertex v where r is placed. Let v' be a vertex adjacent to v . Then $c(v') = c(v) + \left(\frac{k-1}{2} + 1\right) - \frac{k-1}{2}$, where $\frac{k-1}{2} + 1$ is the contribution of the robots closer to v than v' , and $-\frac{k-1}{2}$ is the (negative) contribution of the robots farther from v than v' . Then $c(v) < c(v')$, and as the centrality of a vertex increases with the distance from v , the first part of the lemma is shown.

If k is even, let u and v be the two vertices where the central robots lie. In the degenerated case where $u = v$ it is easy to prove that u is a WP as the centrality of an adjacent vertex u' is $c(u) + \left(\frac{k-a}{2} + a\right) - \frac{k-a}{2}$, where a is the number of robots on u , and the centrality of farther vertices increases.

If u and v are adjacent, $c(u) = c(v) + \frac{k}{2} - \frac{k}{2}$, where $\frac{k}{2}$ is the positive (negative, resp.) contribution of robots closer (farther) to v than u . Then $c(u) = c(v)$, and it is not difficult to see that the centrality of other vertices is larger.

If u and v are not adjacent, let u' be the vertex adjacent to u in the path to v . Then $c(u) = c(u')$ since, as above, $c(u) = c(u') + \frac{k}{2} - \frac{k}{2}$. This is the same for all the vertices in the path including v . For vertices not in the path the centrality is at least $c(v) + \left(\frac{k-b}{2} + b\right) - \frac{k-b}{2}$, where b is the number of robots on v . $\square$

Theorem 6. *Optimal gathering on unidimensional grids is always achievable but for configurations with only two robots or admitting partitive automorphisms.*

Proof. The ungatherable cases simply follow from Theorem 2 and Theorem 3.

When the number of robots is odd, from Lemma 1 there exists only one WP and optimal gathering can be achieved by Corollary 1.

When the number of robots is even, if the configuration is symmetric, then the subpath of WPs must be odd as otherwise the configuration is partitive. The idea is then to let move the robots delimiting the WPs toward the central vertex. If both move synchronously, the configuration remains symmetric but the interval of WPs is reduced until only the WP at the central vertex remains. If only one moves, it is possible to recognize the robot that has to move to (re)-establish the symmetry. In fact, considering the two intervals of free vertices neighboring the robots delimiting the WPs, the algorithm allows to move the robot delimiting the shortest interval.

When the number of robots is even, but the configuration is asymmetric, then either it is at one move from a possible symmetry which is allowed, or one of the two robots delimiting the WPs can be chosen to move toward the other one without creating a symmetry until only one WP remains.

Finally, when there is only one WP, from Corollary 1, all robots can move safely toward it. $\square$

It is worth noting that the algorithm provided by the proof of Theorem 6 also works when the input configuration admits multiplicities.

3.3. Bidimensional grids

In this section, we provide a general optimal algorithm to solve the gathering problem for each configuration $\mathcal{C} = (G, \ell)$ such that $\mathcal{C} \notin \mathcal{U}$. From Corollary 1, if the configuration $\mathcal{C}$ admits only one WP (that is, $\mathcal{C} \in \mathcal{S}$), then optimal gathering can be accomplished. Another characterization is provided by considering $S_{\mathcal{C}}$, and in particular the projections of the robots to the two generating paths P_1 and P_2 of G . Given a robot on a generic vertex (i, j) of G , its projections on P_1 and P_2 are a robot on vertex i and a robot on vertex j , respectively. This gives rise to two configurations (P_1, ℓ_1) and (P_2, ℓ_2) such that $\ell_1(v) = \sum_j \ell((v, j))$ and $\ell_2(v) = \sum_i \ell((i, v))$. As the movements on a grid are either vertical or horizontal, solving the gathering with respect to the two dimensions separately, solves the general problem.

Proof. For each side of S_C , consider the robot (or the two robots) farthest from the central vertex of the chosen side. The algorithm first makes all other robots move toward the center of S_C . At this point, we consider different cases according to the number of robots occupying the corners of S_C :

No corners occupied. Each robot lying on a side of S_C is moved to the center of the side. The obtained configuration is then composed of one or two robots at the center of each side and zero or more robots at the center of S_C .

One corner occupied. Since one corner is occupied, the robots (if any) on the sides defining the occupied corner have been moved by the algorithm toward the center of S_C . For each of the other two sides, let r be the closest robot to the occupied corner, the algorithm makes the other robot (if any) move toward the center of S_C . Then, r is moved to the center of its side. The obtained configuration is then composed by the robot on the corner, no other robots on the two sides sharing one coordinate with the corner occupied, one robot for each of the other two sides at their centers, and at least three robots in the center of S_C .

Two corners occupied. If the two corners occupied do not share any coordinate, then the algorithm makes all the robots move but those in the corners toward the center of S_C , where there will be at least four robots.

If the two corners occupied share one coordinate, then consider the only side s of S_C whose extremities are not occupied. All the robots lying on s but the farthest one(s) are moved toward the center of S_C . After, the algorithm makes the remaining robot(s) on s move to its center. The obtained configuration is then composed by the two robots on the corners, no other robots on the three sides sharing one coordinate with the corners occupied, one robot or two robots in the center of s , and at least two robots in the center of S_C .

Three corners occupied. In this case, there are two robots in two different corners not sharing any coordinate, hence the algorithm makes all the other robots move toward the center of S_C , where there will be at least four robots.

Four corners occupied. The algorithm makes all robots move but those in the corners toward the center of S_C , where there will be at least two robots.

In all the above cases, the obtained configuration admits only one WP at the center of S_C . This can be easily checked considering (G, ℓ) as the Cartesian product of two configurations on two paths (P_1, ℓ_1) and (P_2, ℓ_2) where $\ell_1(x) = \sum_y \ell((x, y))$ and $\ell_2(y) = \sum_x \ell((x, y))$, and by applying [Lemma 1](#) and [Theorem 8](#).

By [Corollary 1](#), at this point all the robots can move toward the only WP, hence achieving the gathering. Moreover, as all the computed moves have been performed toward the final WP, then the proposed algorithm is optimal. $\square$

For the case of just 4 robots, the next lemma holds.

Lemma 3. Let $C = (G, \ell)$ be an initial configuration inducing S_C with both sides odd and the center being a WP, then if there are only 4 robots, there exists an optimal gathering algorithm unless each robot occupies a different corner of S_C .

Proof. By [Theorem 4](#), the configuration with the four corners of S_C occupied is ungatherable. We consider different cases according to the number of robots occupying corners of S_C .

No corners occupied. In this case, the four robots must lie on the border of S_C , one for each side. For each side, the algorithm makes the corresponding robot move toward its center.

One corner occupied. In this case, either there is one side of S_C with two robots, or there is a robot not lying on the border of S_C . In the former case, let r be the robot closest to the occupied corner but the one on the corner itself among the two on the same side of S_C . In the latter case, let r be the internal robot. The algorithm makes r move toward the center of S_C . Then, each robot on one side of S_C but the one in the corner is moved toward the center of the side.

Two corners occupied. If the two corners occupied do not share any coordinate, then the algorithm makes the two robots not occupying corners move toward the center of S_C , where there will be two robots.

If the two corners occupied share one coordinate, then consider the only side s of S_C whose extremities are not occupied. If only one robot lies on s , then the fourth robot is moved toward the center of S_C , and then the robot lying on s is moved to its center. If two robots lie on s and the configuration is asymmetric, then the robot farthest from the center of s is moved toward the center of S_C , while the other is moved to the center of s . In both cases, the same configuration is reached where the set of WPs is constituted by the vertices on the shortest path from the center of S_C to the center of the side shared by the occupied corners. After, the robot lying in the center of S_C moves to the center of the side shared by the occupied corners where there will remain a unique WP.

If two robots lie on s and the configuration is symmetric, then those two robots are moved toward the center of s . Note that, from symmetric configurations, the two robots allowed to move can generate a configuration where only one robot has moved, then the other robot either will (re)-establish the symmetry, or it applies the strategy of the asymmetric case, eventually.

Only from symmetric configurations where both robots always make the allowed move, it is possible to reach a configuration with the two corners occupied and the other two robots at the center of s . In this case, the gathering is accomplished by letting move the robots from the corners toward the multiplicity without creating another multiplicity.

Three corners occupied. In this case, there are two robots in two different corners not sharing any coordinate, hence the algorithm makes the other two robots move toward the center of S_C , where there will be two robots.

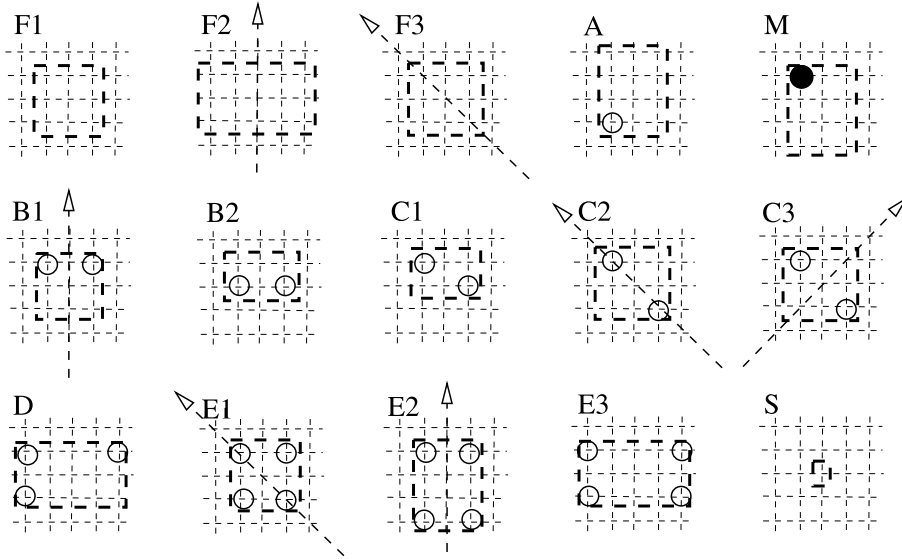


Fig. 3. Types of configurations according to the number of corners of $G_{WP}(C)$ occupied by robots/multiplicities. Dashed lines delimit $G_{WP}(C)$, empty circles represent single robots, and filled circles represent multiplicities.

Similarly to Lemma 2, in all the above cases, the obtained configuration admits only one WP and by Corollary 1 optimal gathering can be assured. Moreover, as all the computed moves have been performed toward the final WP, then the proposed algorithm is optimal. $\square$

Remark. Configurations admitting rotations but not in $\mathcal{U}$ are solved by the above two lemmata since the center of rotation is a vertex in such cases (i.e., S_C has both sides odd) and it is a WP.

3.3.3. Grids with an even number of robots, S_C with a side even or its center is not a WP

Before proceeding with the characterization of the algorithm, we need to better specify the view of the robots during their Look phase when no multiplicities occur.

Let us consider the eight sequences of distances (number of empty vertices) between occupied vertices obtained by traversing S_C starting from its four corners and proceeding toward the two possible directions. If proceeding vertically, all columns will be considered sequentially. Similarly for the rows if proceeding horizontally. Note that the two sequences associated to a corner occupied by a robot start with 0. For instance, by referring to Fig. 2, the upper leftmost corner is associated with (6, 7, 1, 18, 43, 28, 9, 19, 5) by reading the configuration horizontally, and with (1, 27, 24, 6, 36, 15, 11, 12, 4) by reading the configuration vertically.

We associate for each corner the lexicographically largest sequence between the two readings from such corner. Note that, in square grids such two sequences are always different, but for the two corners through which a possible axis of symmetry passes. In rectangular grids, if the two sequences are equal, we assume larger the one read in the direction of the largest side.

We define the maximal sequence as the largest one among the four sequences associated to the four corners. We refer to the corner(s) defining the maximal sequence as *preferred corner(s)*, and to the direction(s) that implies the maximal sequence as *preferred direction(s)*.

In Fig. 2, the preferred corner of S_C is the bottom-leftmost one, the preferred direction is horizontal, and the maximal sequence is (10, 11, 21, 34, 19, 12, 21, 7, 1).

We are now ready to describe the gathering algorithm for each configuration $C \notin \mathcal{U}$ with more than one WP, where S_C has at least one side even or its center is not a WP.

In general, if a configuration is symmetric, the algorithm may allow to move two symmetric robots. If both move, the configuration remains symmetric. If only one moves, the algorithm always forces to move the one that can (re)-establish the symmetry. In fact, we prove that from asymmetric configurations at one step from an allowed symmetry, it is always possible to detect one unique robot that has to move in order to (re)-establish the symmetry.

Let C be a configuration. According to the number of corners of $G_{WP}(C)$ occupied by robots, different strategies are applied. See Fig. 3 for a visualization of the configurations that will be considered.

Type F: no corners occupied Among these configurations, F1 are the asymmetric ones, F2 the symmetric configurations with an horizontal/vertical axis, and F3 the symmetric configurations with a diagonal axis.

First we consider the cases when $G_{WP}(C)$ is not a path.

If C is in $F2$, then among the robots determining $G_{WP}(C)$, consider those closest to $G_{WP}(C)$. Among such robots consider those closest to $G_{WP}(C)$ with respect to the preferred direction, if any. Only two robots are then selected by considering those closest to the preferred corners. Such robots move toward $G_{WP}(C)$. In this case, either two symmetric robots move synchronously, or only one moves, and the other one is possibly pending. We will show that our algorithm always force the possible pending robot to move. Eventually, this process leads to symmetric configurations with two corners of $G_{WP}(C)$ occupied and the axis reflecting them.

If C is in $F3$, then among the robots determining $G_{WP}(C)$, consider those closest to $G_{WP}(C)$. Among such robots consider those closest to $G_{WP}(C)$ with respect to the preferred direction, if any. Only two robots are then selected by considering those closest to the preferred corner(s). Such robots move toward $G_{WP}(C)$. As above, our algorithm forces the two selected robots to move symmetrically. Again, we will show that our algorithm always force the possible pending robot to move. Eventually, this process leads to symmetric configurations with one corner occupied by a multiplicity.

If C is in $F1$ at more than one move from an allowed symmetry, the closest robot to $G_{WP}(C)$ moves toward it. Ties are solved by considering the preferred direction and the preferred corner.

When $G_{WP}(C)$ is constituted by just a path, the strategy is the same as above but limited to the robot(s) lying on the extension of $G_{WP}(C)$, and not those determining its extremities.

Type A: one corner occupied by a single robot If the configuration is asymmetric, first robots check whether an allowed symmetry can be (re)-established. We will show that this is possible by finding the previous positions of the unique robot on the corner of $G_{WP}(C)$. If a symmetry can be (re)-established, then the possible pending robot is forced to perform its move. In any other case, the single robot on the corner moves in one of the two directions that reduce $G_{WP}(C)$, until obtaining only one WP.

Type M: one corner occupied by a multiplicity and possibly other corners occupied First, all robots sharing one coordinate with the multiplicity move in turn (i.e., without creating another multiplicity) until joining it, toward the shared coordinate. After that, if there are more than one WP, among the robots determining $G_{WP}(C)$, consider those closest to it. Such robots – at most four – move (even concurrently) along the direction that reduces $G_{WP}(C)$, until they share one coordinate with the multiplicity.

Types B and C: two corners occupied Among these configurations we denote by B the set of configurations where the two corners occupied share one coordinate but for symmetric configurations with the axis passing through the two occupied corners. These last configurations and the remaining ones with two corners occupied are denoted by C . $B1 \subset B$ represents symmetric configurations, $B2 \subset B$ the asymmetric ones. $C1 \subset C$ represents asymmetric configurations, $C2 \subset C$ the symmetric ones with the axis passing through the occupied corners, and $C3 \subset C$ the remaining symmetric configurations.

From $B1$, the two robots on $G_{WP}(C)$ move toward each other, still maintaining the symmetry. Note that, the two robots are separated by an odd path, as otherwise the configuration is ungatherable by [Theorem 2](#).

From $B2$, the algorithm (re)-establishes an allowed symmetry, if any. Otherwise, the robot r on the corner of $G_{WP}(C)$, closest to the preferred corner, moves toward the other robot r' . If such a move would generate an ungatherable configuration, then r' moves toward r .

From $C1$, the algorithm (re)-establishes an allowed symmetry, if any. Otherwise, if $G_{WP}(C)$ is composed of more than four vertices, the first robot r met among the two on the corners of $G_{WP}(C)$ from the preferred corner along the preferred direction moves toward the other robot r' unless it makes $G_{WP}(C)$ as a path or the move brings to an ungatherable configuration. In which case, r' moves. When $G_{WP}(C)$ is a squared grid of four vertices, then we ensures that the selected robot can safely move toward the other one.

From $C2$, if $G_{WP}(C)$ is not a path, the robot that moves is the one on the corner of $G_{WP}(C)$ closest to the corner of S_C associated with the maximal sequence among the two corners on the axis. If $G_{WP}(C)$ is a path, the robot that moves is the one closest to the corners of S_C associated with the maximal sequence.

From $C3$, the two robots on the corners of $G_{WP}(C)$ move toward the corner of $G_{WP}(C)$ on the axis, closest to the corner of S_C associated with the maximal sequence.

In all the cases, a configuration in S with a multiplicity occupying the only WP will be reached, eventually.

Type D: three corners occupied From D , the robot on the middle corner moves toward one of the two other occupied corners. This leads to a configuration with two corners occupied or with one corner occupied by a multiplicity. We will show that if the configuration is at one move from an allowed symmetry, then the defined move involves exactly the robot that can (re)-establish the symmetry.

Type E: four corners occupied Among configurations E , we denote by $E1$ the symmetric ones with a diagonal axis, by $E2$ the remaining symmetric ones, and by $E3$ the asymmetric ones.

From $E1$, the robot on $G_{WP}(C)$ closest to the corner of S_C on the axis associated with the maximal sequence, moves reducing the WPs.

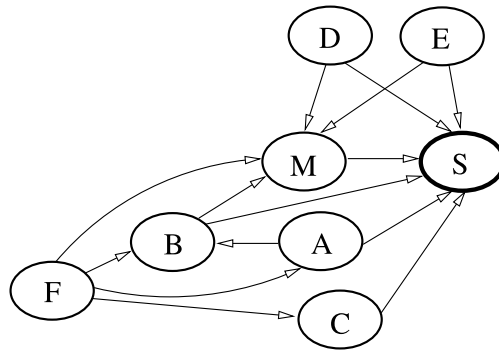
From $E2$, the two robots on $G_{WP}(C)$ closest to the preferred corners move toward each other. Note that, if both move synchronously, then a symmetric configuration with two corners occupied or with a multiplicity is obtained. If only one

```

Procedure: COMPUTE-PHASE
Input:  $C = ((V, E), \ell)$ 
1 Compute  $S_C$ ,  $G_{WP}(C)$ , and  $k = \sum_{v \in V} \ell(v)$ ;
2 if  $C \notin \mathcal{U}$  then
3   if  $C \in S$  then
4     any robot moves towards the unique WP;
5   else
6     if  $S_C$  has both sizes odd and its center is a WP then
7       if  $k = 4$  then Apply the strategy defined by Lemma 3;
8       else Apply the strategy defined by Lemma 2;
9     else
10      case  $C \in \mathcal{X}$ , with  $\mathcal{X} \in \{A, B, C, D, E, F, M\}$ 
11      do Apply the strategy designed for type  $\mathcal{X}$ ;

```

Fig. 4. Procedure COMPUTE-PHASE.

Fig. 5. Transitions among types of configurations allowed by the optimal gathering algorithm when S_C has at least one side even or its center is not a WP.

moves, then a configuration of type D is obtained. According to that case, the robot that will be allowed to move is the one with a possible pending move, hence it does not create further pending moves.

From $E3$, the robot on $G_{WP}(C)$ closest to the preferred corner moves reducing the WPs.

Summarizing, the algorithm applied by the robots during the compute phase is shown in Fig. 4.

4. Correctness

In order to prove the correctness of the proposed algorithm, we remark that when $C \in S$, the correctness is guaranteed by Corollary 1. By Theorem 8, S includes all configurations with an odd number of robots. When there are more than one WP, if C contains an even number of robots with S_C admitting both sides odd and its center being a WP, the correctness is guaranteed by Lemmata 2 and 3.

When S_C has at least one side even or its center is not a WP, we are going to prove that the defined strategy always leads to configurations in S , and from there the gathering is finalized by applying Corollary 1. Moreover, along this transition, the target vertex where gathering is finalized never changes, hence robots always move along their shortest paths toward the gathering vertex.

The general scheme of the transitions obtainable by our algorithm when S_C has at least one side even or its center is not a WP is shown in Fig. 5. For the ease of visualization, self-loops are not shown in the figure since we prove that from each class but S , an exit transition is always taken, eventually. From S , instead, gathering is always finalized. We have to prove that the only possible transitions generated by our algorithm are those depicted in the figure, hence being S the only sink node that will be reached, eventually.

Lemma 4. *From an asymmetric configuration C with no corners of $G_{WP}(C)$ occupied (F1), there is at most one pending move that can be detected.*

Proof. If the configuration is at one move from a solvable symmetry, according to the described strategy for symmetric configurations in F , there must be exactly one robot r closest to $G_{WP}(C)$ and determining one of its sides, as otherwise we are sure there is no pending move. If such, the algorithm evaluates the previous position of r by considering the configuration with r at one step farther from $G_{WP}(C)$, and there is only one direction to check (the one determining one

side of $G_{WP}(C)$). If this test gives a symmetric and solvable configuration, then the algorithm allows to move the symmetric robot in the specular direction. In doing so, the possible correct symmetry is (re)-established. $\square$

In general, all moves are performed toward $G_{WP}(C)$, and in particular toward the final gathering vertex. This implies that the gathering is achieved optimally.

Lemma 5. *From configurations in F , only configurations in A with a possible pending move, in $B1$, in M or in $C3$ are reachable within a finite number of moves.*

Proof. By Lemma 4, the moves specified by the algorithm bring one or two robots on the corners of $G_{WP}(C)$. In particular, from $F1$ it is possible to reach $F1$ if the configuration is not at one step from a solvable symmetry; $F2$ or $F3$ if an allowed symmetry can be (re)-established; A if the only robot allowed to move reaches one corner of $G_{WP}(C)$.

From $F2$ it is possible to reach $F1$ if only one robot moves and then again $F2$ by Lemma 4; A with a possible pending move if only one robot moves and reaches a corner of $G_{WP}(C)$; $B1$ if both robots move and reach the corners of $G_{WP}(C)$;

From $F3$ it is possible to reach $F1$ if only one robot moves and then again $F3$ by Lemma 4; A with a possible pending move if only one robot moves and reaches a corner of $G_{WP}(C)$; $C3$ if both robots move and reach two different corners of $G_{WP}(C)$; M if both robots move and reach the same corner of $G_{WP}(C)$.

In all the cases, the distance to $G_{WP}(C)$ of the moved robots is decreased. Moreover, when $G_{WP}(C)$ is not a path and only one robot moves, it is not possible to reach a symmetric configuration unless a configuration in A is reached. This is due to the fact there is no other robot at such distance among those determining the border of $G_{WP}(C)$. Hence the only case in which the reached configuration is symmetric occurs when the moved robot lies on one corner of $G_{WP}(C)$ and hence we need to show it is not in $\mathcal{U}$. Since only one corner of $G_{WP}(C)$ is occupied, the only admissible axis of symmetry is the diagonal one passing through the occupied corner, that is the implied automorphism is not partitive.

When $G_{WP}(C)$ is a path and only one robot moves, the configuration obtained is not in $\mathcal{U}$ as the only possible axis of symmetry passes over all $G_{WP}(C)$, hence it is not partitive. $\square$

Theorem 9. *From a configuration C in A with a possible pending move, only configurations in $B1$ or S are reachable.*

Proof. Let C' be the configuration from which C has been obtained by applying our algorithm. Let r be the only robot occupying a corner of $G_{WP}(C)$.

If $G_{WP}(C)$ is not a path and C' is symmetric, then according to our algorithm C' induces the same set of WPs of C unless r was lying on a diagonal axis of symmetry. In this last case, there are no pending moves, hence applying the algorithm from C , a new configuration of type A or S is obtained but with less WPs than C . In the first case there can be a possible pending move. The two possible previous positions of r can potentially induce two orthogonal axes of symmetries. It follows that in order to be recognized as gatherable configurations by our algorithm, both axes must pass through vertices, hence inducing S_C with both sides odd with a WP in the center, but this is managed by Lemmata 2 and 3. Hence, only one previous position of r can result in having an allowed symmetric configuration, and in such a case, the symmetric robot to r is moved specularly to (re)-establish the symmetry, leading to a configuration in $B1$.

If C' is asymmetric, instead, there are no pending moves and by applying our algorithm from C , a new configuration of type A or S is obtained but with less WPs than C .

If $G_{WP}(C)$ is a path, then either the configuration has been obtained by moving r along the direction of the path, or orthogonally.

In the latter case, $G_{WP}(C')$ can be either a rectangular grid or simply a square grid of four vertices. In either cases there are no pending moves, since the only possibility for C' to be symmetric may occur when $G_{WP}(C')$ is squared and admits a diagonal symmetry with the axis passing through r .

In the former case, C' can be symmetric. If the axis passes through all vertices of $G_{WP}(C')$, then there are no pending moves in C . If the axis cuts $G_{WP}(C')$ in its middle which must be a vertex, then the symmetric robot to r in C' is moved to (re)-establish the symmetry. In all the other cases, r moves reducing the number of WPs. $\square$

Theorem 10. *From a configuration C in B , only configurations in M or S are reachable.*

Proof. If $C \in B1$, the two robots on the corners of $G_{WP}(C)$ move toward the central WP on the shortest path connecting them. If both move, the obtained configuration is still in $B1$ or in M or, if $G_{WP}(C)$ is a path, in S . If only one moves, the obtained configuration is in $B2$ with a possible pending move.

If $C \in B2$, the algorithm first checks whether C could have been obtained from an allowed symmetric configuration with a possible pending move, in which case it forces to (re)-establish such a symmetry as follows. When $G_{WP}(C)$ is composed by more than two WPs, let r be the robot occupying the corner of $G_{WP}(C)$, furthest from the corners of S_C . If, by considering r on its previous position (along the unique possible direction), an allowed symmetry is obtained, then the specular robot to r (if any) is moved to (re)-establish the symmetry, hence reaching a configuration in $B1$ or M .

When $G_{WP}(C)$ is composed of just two WPs, then we have to carefully analyze how to detect possible pending moves. Let r_1 and r_2 be the two robots on $G_{WP}(C)$, and refer to C_1 and C_2 as the configurations obtained by considering r_1 and r_2 on their previous positions, respectively. There are potentially, three previous positions for each robot. We first consider the case whether both C_1 and C_2 are symmetric. Without loss of generality, we assume r_1 and r_2 horizontally disposed. Let C' be the configuration obtained from C by removing r_1 and r_2 . Note that the number of robots in C' is not null as we are considering configurations not in $\mathcal{U}$, and hence with more than two robots.

Consider both r_1 and r_2 on their horizontal previous positions, then C_1 and C_2 could admit a vertical axis of symmetry or a rotational symmetry of 180 degrees centered in the center of $G_{WP}(C_1)$ or $G_{WP}(C_2)$. Then C' must admit two different vertical reflections, or one vertical reflection and one rotation, or two different rotations. In all cases, C' should admit an infinite number of robots or no robots at all.

If r_1 is considered on its horizontal previous position and r_2 on one of its vertical previous position, then C_1 could admit a vertical reflection or a rotation and C_2 could admit a diagonal axis of symmetry. If C_1 is rotational, as the center must be on the center of $G_{WP}(C_1)$, then both the sides of S_C are odds, and this is a contradiction. So C_1 could admit only a vertical reflection. In this case, as we are assuming that C_2 has a diagonal axis of symmetry, C' is rotational with the center on a vertex and hence both the borders of S_C are odds. Note that the borders of S_C and $S_{C'}$ are the same, as r_1 and r_2 can not be on the borders of S_C . In fact, if r_1 (r_2 , resp.) is on the border of S_C , the vertical (diagonal) symmetry of C_1 (C_2) and the fact that r_2 (r_1) is a WP in C_1 (C_2) would imply that there are no other robots but r_1 and r_2 .

Considering both r_1 and r_2 on one on their vertical previous positions, then both C_1 and C_2 could admit only diagonal symmetries. As above, we can assume that r_1 and r_2 are not on the borders of S_C . If a diagonal passes through both r_1 and r_2 in C_1 (in C_2 , resp.) then $C_1 \in C_2$ ($C_2 \in C_1$), so by following the algorithm, in C there are no pending moves, and then the possible pending move can be performed due to the symmetry in C_2 (C_1), reaching a configuration in S . If the two axes do not pass through r_1 and r_2 , then C' is rotational with a center not on a vertex. So both C and C_2 admit a partitive automorphism, impossible. The case in which the diagonal axis of C_1 and that of C_2 are parallel is also not possible as C' should admit a translation hence implying an infinite number of robots or no robots at all, against the hypothesis.

We have shown that either two distinct robots cannot (re)-establish distinct symmetries or there are no pending moves. It remains to consider the case in which one robot, say r_1 , can (re)-establish two (or more) symmetries by means of different previous positions. But in this case the only pending move could be that of r_2 , then the algorithm moves r_2 toward r_1 and a configuration in S is reached.

If by considering possible previous positions for r_1 or r_2 no allowed symmetries can be obtained, then there are no pending moves. Without loss of generality, let r_1 be the robot closest to the preferred corner along the preferred direction. If by moving r_1 toward r_2 leads to a configuration not in $\mathcal{U}$ the move is allowed, otherwise r_2 is moved toward r_1 . In this case, by similar arguments as above, the obtained configuration cannot be in $\mathcal{U}$.

In all the cases, each move performed by the robots reduces the number of WPs. Hence, if a configuration remains in B after a move, by induction a configuration in M or S will be reached, eventually. $\square$

Theorem 11. *From a configuration C in C , only configurations in S are reachable.*

Proof. If $C \in C_2$ and $G_{WP}(C)$ is not a path, the defined move leads to a configuration in C_1 without pending moves. If $G_{WP}(C)$ is a path, the defined move leads to a configuration in C_2 with a smaller number of WPs or in S . Ungatherable configurations cannot be generated as the maximal sequence of the preferred corner has been enlarged, and hence no other symmetries can be obtained.

If $C \in C_1$, we need to check whether there might be pending moves, i.e., if the configuration has been potentially obtained from a symmetric one. To this respect, $G_{WP}(C)$ cannot be a square grid, and its sides must differ of exactly one. It follows that we need to check the previous position of each robot along the only direction that makes squared the grid of WPs. If the configuration obtained by considering one of such robots on its previous position is in C_2 , then we know there are no pending moves hence we can consider the other robot and check whether it may have come from a configuration in C_3 . The case where both robots may have moved from a configuration in C_3 is not possible, since the configuration obtained from C by removing the two robots from the two corners of $G_{WP}(C)$ should admit a translation. So, if one robot may have come from different configurations in C_3 , the algorithm moves the other one to (re)-establish the symmetry. If no symmetries can be (re)-established and $G_{WP}(C)$ is composed of more than four vertices, the algorithm makes move the first robot on the corner of $G_{WP}(C)$ met in the maximal sequence, toward the other one without making $G_{WP}(C)$ a path and without creating ungatherable configurations. If this is not possible, then the algorithm makes the other robot on the corner of $G_{WP}(C)$ move. In this latter case, the move is always possible because no further symmetries nor rotations can be created for similar arguments of Theorem 10. This is repeated until a configuration of type C_2 , C_3 with less WPs, or C_1 with $G_{WP}(C)$ composed of four vertices is reached. When $G_{WP}(C)$ is made of just four corners, it is always possible for the robot r_1 lying on the corner of $G_{WP}(C)$ met as first in the maximal sequence to choose one move without incurring in ungatherable configurations. In fact, let C_1 and C_2 be the two configurations obtained by moving r_1 toward the other robot

in the two possible directions, respectively. If C_1 or C_2 admit a reflection with the axis passing through r_1 , then the move can be performed.

Assuming both C_1 and C_2 admit a reflection with the axis cutting the edge between the WPs, then C must be rotational with the center not on a vertex, contradicting the asymmetry of C .

Assuming both C_1 and C_2 admit a rotation with the center on the edge between the WPs, then C without r_1 and r_2 must admit two different rotational centers, implying no robots, or an infinite number of robots.

Without loss of generality, assuming C_1 admits a reflection with the axis cutting the edge between the WPs and C_2 admits a rotation with the center on the edge between the WPs, again C without r_1 and r_2 must be composed of no robots, or an infinite number of robots.

Hence, the obtained configuration has only two WPs occupied by r_1 and r_2 , and it is either symmetric of type C2 or asymmetric of type B2 without pending moves. By the move defined for configurations in C2, a configuration in S is obtained. Similarly, by Theorem 10, a configuration in S is reached from B2.

If $C \in C3$, the defined move reduces $G_{WP}(C)$ until the configuration belongs to S . If only one robot moves, then the obtained configuration is either in C1 or in the degenerate case B with the area of WPs made of just two robots. In the first case, the possible pending move is managed as described above. In the second case, the possible pending move is managed as described in Theorem 10. $\square$

Theorem 12. *From configurations in D , only configurations in M or S are reachable.*

Proof. In order to check whether there is a symmetry to (re)-establish it is enough to consider that a configuration in D can be obtained only from one in E . According to the algorithm, the only pending move can refer to the robot occupying the central corner among the three occupied. Since the algorithm moves exactly such a robot, it does not generate further pending moves and the reached configuration is either in B or in M . In the first case, by Theorem 10, configurations in M or S will be reached, eventually. $\square$

Theorem 13. *From configurations in E , only configurations in M or S are reachable.*

Proof. From a configuration in $E1$, the defined move leads to a configuration in D , and by Theorem 12 the claim holds. From a configuration in $E2$, either two robots move synchronously leading to B , or only one moves leading to D . In either cases, by Theorem 10 and Theorem 12 the claim holds. From $E3$, robots can deduce that this is necessarily an initial configuration, hence the robot on the corner associated with the maximal sequence moves, leading to D or to M . Hence, again by Theorem 12 the claim holds. $\square$

Theorem 14. *From a configuration $C \in M$, only configurations in S are reachable.*

Proof. From M , according to the first part of the defined strategy, all robots sharing one coordinate with the multiplicity will join it, eventually, hence remaining in M or reaching S . If the configuration is still in M , at most four robots can move concurrently. These are the robots not lying in the multiplicity that determine $G_{WP}(C)$. They are at most two for each side of $G_{WP}(C)$ not sharing coordinates with the multiplicity. The aim is to reach a configuration where such robots share one coordinate with the multiplicity. Once this happens, the configuration is in S . Before that, even if robots move synchronously, no other multiplicities can arise, as the moved robots have different target vertices sharing one coordinate with the multiplicity, and such vertices are empty due to the first part of the defined strategy. $\square$

Summarizing, by the obtained results we can state the following theorem.

Theorem 15 (Gathering). *Given an initial configuration $C = (G, \ell)$ on an infinite grid G , optimal gathering can be assured unless $C \in \mathcal{U}$.*

5. Conclusion

We have studied the gathering problem under the Look–Compute–Move model with global-strong multiplicity detection on infinite grids. We have shown how the assumed model represents the minimal setting in order to assure optimal gathering. This means that the number of moves required by the designed algorithm equals the minimal centrality among the vertices of the initial configuration.

We fully characterize optimal gathering in terms of computed moves on infinite grids. This is one of the most important topologies for robot-based computing systems as it represents a natural discretization of the plane. Nonetheless, our algorithm also works for finite grids, hence improving the results on that respect to optimal gathering. Other topologies might be worth investigating such as tori and hypercubes.

References

- [1] G. Di Stefano, A. Navarra, Optimal gathering on infinite grids, in: *Proc. of the 16th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems, SSS*, in: *Lecture Notes in Computer Science*, vol. 8756, 2014, pp. 211–225.
- [2] B. Degener, B. Kempkes, T. Langner, F. Meyer auf der Heide, P. Pietrzyk, R. Wattenhofer, A tight runtime bound for synchronous gathering of autonomous robots with limited visibility, in: *Proc. of the 23rd ACM Symp. on Parallelism in Algorithms and Architectures, SPAA*, 2011, pp. 139–148.
- [3] Y. Dieudonné, A. Pelc, V. Villain, How to meet asynchronously at polynomial cost, in: *Proc. of the 32nd ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, PODC*, 2013, pp. 92–99.
- [4] P. Flocchini, G. Prencipe, N. Santoro, P. Widmayer, Gathering of asynchronous robots with limited visibility, *Theor. Comput. Sci.* 337 (2005) 147–168.
- [5] P. Flocchini, G. Prencipe, N. Santoro, *Distributed Computing by Oblivious Mobile Robots*, *Synthesis Lectures on Distributed Computing Theory*, Morgan & Claypool, 2012.
- [6] T. Izumi, T. Izumi, S. Kamei, F. Ooshita, Randomized gathering of mobile robots with local-multiplicity detection, in: *Proc. of the 11th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems, SSS*, in: *Lecture Notes in Computer Science*, vol. 5873, 2009, pp. 384–398.
- [7] G. D'Angelo, G. Di Stefano, A. Navarra, Gathering asynchronous and oblivious robots on basic graph topologies under the look–compute–move model, in: *Search Theory: A Game Theoretic Perspective*, Springer, 2013, pp. 197–222.
- [8] S. Devismes, A. Lamani, F. Petit, P. Raymond, S. Tixeuil, Optimal grid exploration by asynchronous oblivious robots, in: *Proc. of the 14th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems, SSS*, in: *Lecture Notes in Computer Science*, vol. 7596, 2012, pp. 64–76.
- [9] R. Klasing, A. Kosowski, A. Navarra, Taking advantage of symmetries: gathering of many asynchronous oblivious robots on a ring, *Theor. Comput. Sci.* 411 (2010) 3235–3246.
- [10] R. Klasing, E. Markou, A. Pelc, Gathering asynchronous oblivious mobile robots in a ring, *Theor. Comput. Sci.* 390 (2008) 27–39.
- [11] I. Suzuki, M. Yamashita, Distributed anonymous mobile robots: formation of geometric patterns, *SIAM J. Comput.* 28 (4) (1999) 1347–1363.
- [12] G. D'Angelo, G. Di Stefano, A. Navarra, Gathering on rings under the look–compute–move model, *Distrib. Comput.* 27 (4) (2014) 255–285.
- [13] G. D'Angelo, G. Di Stefano, R. Klasing, A. Navarra, Gathering of robots on anonymous grids and trees without multiplicity detection, *Theor. Comput. Sci.* 810 (2016) 158–168.
- [14] G. Di Stefano, A. Navarra, Optimal gathering of oblivious robots in anonymous graphs, in: *Proc. of the 20th Int. Colloquium on Structural Information and Communication Complexity, SIROCCO*, in: *Lecture Notes in Computer Science*, vol. 8179, 2013, pp. 213–224.
- [15] Y. Kupitz, H. Martini, Geometric aspects of the generalized Fermat–Torricelli problem, in: *Intuitive Geometry*, vol. 6, in: *Bolyai Society Math. Studies*, 1997.
- [16] J. Chalopin, P. Flocchini, B. Mans, N. Santoro, Network exploration by silent and oblivious robots, in: *Proc. of the 36th Int. Workshop in Graph Theoretic Concepts in Computer Science, WG*, in: *Lecture Notes in Computer Science*, vol. 6410, 2010, pp. 208–219.
- [17] C. Bajaj, The algebraic degree of geometric optimization problems, *Discrete Comput. Geom.* 3 (1) (1988) 177–191.
- [18] E.J. Cockayne, Z.A. Melzak, Euclidean constructibility in graph-minimization problems, *Math. Mag.* 42 (4) (1969) 206–208.